

Optimization of Quality of Service in Wireless Sensor Networks Powered by Solar Cells

Soledad Escolar^a, Stefano Chessa^{b,c} and Jesus Carretero^a

^a Computer Science Department. University Carlos III of Madrid. Madrid, Spain.

^b Computer Science Department. University of Pisa. Pisa, Italy.

^c ISTI-CNR. Pisa, Italy.

Email: mariasoledad.escolar@uc3m.es

Abstract

Sensors equipped with solar cells and rechargeable batteries are useful in many outdoor, long-lasting applications. In these sensors the cycles of energy harvesting and battery recharge need to be managed appropriately in order to avoid sensor unavailability due to energy shortages. We suggest that adapting the sensor duty cycle (and specifically its environment sampling frequency) to the expected energy production and residual battery charge is very useful to avoid sensors unavailability. To this purpose we introduce a novel concept of QoS, which is measured in terms of sampling frequency, and we provide a mean to maximize the QoS, i.e. the extent of the period in which the sensor operates at the user's desired sampling frequency.

1 Introduction

Wireless sensor networks (WSN) are composed of wireless embedded devices called sensors capable to monitor in real-time different environmental phenomena through transducers, and to forward the measured data to a special sensor called gateway or sink, where data are collected for further processing.

A sensor is typically powered by limited-capacity batteries. The batteries feed the sensor's circuitry by providing the current draw required to sustain the operation of its components. A sensor integrates several physical components (e.g. radio, transducers, microcontroller, and flash memory), each of which may operate in different states of energy. The sensor energy consumption is then computed as the sum of the individual consumptions of each component at each state; hence, the sensor lifetime is the time taken to discharge its battery below the minimum charge level that is required to sustain the sensor's operation.

To provide potentially infinite sensors' lifetime, recent

studies have investigated the use of energy harvesting to power on the sensors [15]. Unfortunately, in realistic scenarios, energy harvesting is generally intermittent and insufficient to constantly fulfill the needs of the sensors. For this reason the batteries are still necessary: they are recharged when the energy harvesting system produces an excess of energy and they sustain the sensor when the energy production of the energy harvesting is not sufficient. However, the case in which the battery charge is too low and the energy harvesting system does not produce the sufficient energy is always possible and results in (temporary) periods of sensor unavailability. To reduce these periods (and possibly to avoid them), the energy harvesting and battery recharge cycles need to be managed appropriately.

In this paper, we consider WSN for outdoor monitoring applications, where the sensors are exposed to different sources of energy found naturally in the environment, such as the sun, vibration, and acoustic noise. Under this assumption, this work brings together the research areas of green computing and Quality of Service (QoS) of WSN applications. To achieve this, we define an energy harvesting system that exploits the energy obtained from the solar light to sustain the sensor operation and charge its batteries. The main contributions of this paper are two: 1) a system model that allows to express the QoS of the WSN applications from the user perspective; and 2) an application scheduler that optimizes the QoS of the WSN applications accordingly to the estimated energy production from the energy harvesting system. The goal of this scheduler is to maintain the level of service required by the user as long as possible while the energy production allows it. On the other hand, when the estimated energy production is insufficient to maintain the sensor operations at the current level, the system progressively degrades the sensor operation level up to the idle state, that is to say, no operation.

We wish to stress that our view of QoS in WSN is rather different than the usual view of QoS in the literature on

WSN. As the primary goal of a WSN is to ensure a constant monitoring of the environment, we consider the frequency of the environmental sampling the main metric for QoS, and our approach attempts to maximize the extent of the period in which the WSN meets the user's desired sampling frequency, despite energy shortages that may be caused by insufficient energy harvesting. To the best of our knowledge this is the first work addressing QoS at this level and with this objective.

The remainder of this paper is organized as follows: after reviewing the related work in Section II, we present in Section III the design of our energy harvesting system based on a solar cell. Section IV provides the energy production model from the solar cell and presents the realistic battery discharge and charge model. Section V describes our system model to express the QoS of the WSN applications from the user perspective. An example illustrating these concepts is presented in Section VI. Then, in Section VII we propose an approach to efficiently adapt the QoS level of the WSN application on the basis of the solar energy to be produced by the solar cell and, in Section VIII, we provide the simulation results. Finally, Section IX discuss the main conclusions and further research.

2 Related Work

Green computing is a novel general term referring to the capabilities of the Information Technologies (IT) to minimize their negative impact on the environment, especially the CO_2 emissions in the manufacturing process and the energy consumption [6, 14]. In contrast, in WSN energy is the most limited resource, and energy efficiency is not an additional feature of the system, but it is something that must be designed since the beginning in a WSN application and that involves all layers from the hardware to the application.

Dynamic Power Management (DPM) [4] is a technique aimed at energy saving, which consists in switching off the components of a sensor when not necessary and in waking them up on demand. In [18] is proposed a probabilistic energy model to take decisions on when to turn on and off the devices on the basis of event statistics. Sensor components have different energy states and the events with a consumption above a certain threshold allow to transit from one state to another. While the model takes into account the energy overhead due to the components' state switching, it does not rely on a realistic battery discharge model as proposed in [17] and further extended in [16]. In particular, in [17] the authors use the Rakhmatov-Vrudhula battery model instead of a linear discharge model (which is the default) and prolong the battery lifetime by scheduling the transitions to a sleep state after packet transmissions and by taking advantage of the recovery battery effect. To the best of our knowledge neither these works nor other DPM

methods focus on the maximization of the QoS of a WSN application while preserving energy, or take into account a realistic charge and discharge model which is derived from an energy harvesting system.

There exist many works that deal with QoS in WSN [5, 20], but most of them focus on QoS at network or at MAC layers: they aim at defining routing and MAC protocols able to meet the end-to-end QoS requirements while taking into account energy efficiency. A QoS-aware routing protocol not only has to find a route between a source and a destination but it must find the better route according to a QoS metric. Among these metrics, latency is one of the most used [19, 10, 9, 2]. SAR [19] is the first routing protocol for WSN that includes the notion of QoS. In SAR every packet has a priority level and every link has a QoS metric which is computed by taking into account the energy resources and delay. The protocol computes the QoS metric of the route as the product of the additive QoS metric along the path and the priority level of that packet. SPEED [10] and its evolution MMSPEED [9] provide soft real-time delivery guarantee by re-routing packets toward less loaded links. In [2], the problem of finding the optimal path is addressed for WSN real-time applications that use imaging and video sensors. This work aims at finding the lesser delay-constrained path and provides best effort services for throughput. The guarantee of end-to-end bandwidth in wireless ad-hoc networks is the focus of other works in the literature. In [22] the authors provide an algorithm to distributively compute QoS routes with reserved bandwidth in TDMA networks. Similarly, in [12] the problem is addressed for wireless multimedia ad-hoc networks. At the MAC layer, a few MAC protocols address the QoS by providing services differentiation based on static or dynamic criteria and by providing a specific-application QoS support. A survey of these protocols is presented in [21]. A different approach consists of defining an application specific QoS metric, such as coverage, exposure, deployment or reliability. For example, in [11] QoS is defined as the optimum number of sensors that should send data.

Differently than all the above works, in this work we have a different view on QoS in WSN. Specifically, we address the problem at the application level and the sampling frequency is our main metric for QoS. Despite other existing approaches aim to adjust the sampling rates used by energy-hungry sensors [3], any of them is intended to maximize the QoS of the WSN applications.

3 Energy Harvesting System

We have designed an energy harvesting system that produces energy from the sun light. The system is designed to be robust and it includes ultracapacitors and rechargeable batteries that power the sensor in conditions of darkness or

deficient sun light. We establish three priorities of use: by default, the sensor is powered on first directly from the solar cell, second from the ultracapacitors connected in parallel to the cell and third from the two rechargeable batteries. Based on this design we present the requirements of the solar cell and the sensor.

Solar cells are composed of wafers, in the order of micrometers thick, made of some semiconductor material at crystalline state, typically silicon, with the property of converting the energy of the photons in electricity by the photovoltaic effect. The efficiency of a solar cell (η) is a measure of the energy conversion efficiency of the incident light absorbed by a solar cell into electric power, computed as $\eta = \frac{P_{max}}{D \times S}$, where P_{max} is the maximum output power provided by the cell (in Watts), D is the irradiance, defined as the density of incident power on a surface under standard assumptions (in W/m^2) and S is the surface area of the solar cell (in m^2). For this work we selected the MSX-005F solar cell [7] manufactured by BP-SOLAR, a polycrystalline cell that provides a maximum current of 150mA. Table 1 summarizes the electrical specifications of this cell.

Parameter	Value
Specified Load Voltage (V_{ld})	3.3V
Typical Current at Vld (I_{ld})	150mA
Open Circuit Voltage (V_{oc})	4.6V
Short Circuit Current (I_{oc})	160mA
Maximum Output Power (P_{max})	500mW
Surface Area (S)	36cm ²

Table 1. Specifications of MSX-005F solar cell.

The solar cell efficiency depends on the amount of light that it is able to absorb. In turn, this depends on intrinsic factors of the region where it is located: its geographic coordinates, the solar time, its inclination with respect to the sun rays (which should be close to 90 degrees to obtain the maximum efficiency), etc. Consider for example the sun irradiance in Spain, where the system was deployed for some tests. For each Spanish province, given its latitude and longitude, we obtained the average irradiance in each month of the year from RetScreen NASA program [13]. To ensure that the system works in any location, we took the lowest irradiance, which corresponds to the Lugo province (in the North West) during December. Hence, in this case, the efficiency of the solar cell is $\eta=11,38\%$.

To validate our system we use MicaZ [8], a well-known platform for sensor networks applications. Note however, that any other device with a similar energy consumption can be used. MicaZ is composed of a 7 Mhz μ controller, a Zigbee-compliant CC2420 radio, three LEDs, and an external flash memory. To provide capacity of sensing, it may be integrated with different sensor boards. MicaZ is powered

by 2-AA batteries, which should provide a voltage between 2.7 and 3.3 for the correct working of the sensor. According to the documentation, the energy requirements of MicaZ are shown in Table 2.

Parameter	Value			
External Power	2.7V-3.3V			
μ controller	Active		Idle	
	8mA		< 15 μ A	
Radio	Tx	Rx	Idle	Sleep
	17.4 mA	19.7mA	20 μ A	1 μ A
Transducers	On		Off	
	0.7mA		0.0mA	
Flash	Standby	Read	Write	Load
	2 μ A	4mA	15mA	2 μ A
LEDS	On		Off	
	22mA		0.0mA	

Table 2. MicaZ energy requirements.

The power demand of MicaZ depends on the components that need to be on in the application and in which mode. In the worst case, if we take into account the maximum current draw of each component (whose sum is 65.4mA according to Table 2), the power required by MicaZ is $65.4 \times 3.3=251.82mW$. However, it is typically sufficient to guarantee 100mW to power the device.

Knowing the energy requirements of all components and the I_{ld} provided by the cell, we estimate that nearly 100mA are needed to charge the batteries of the sensor. With this current and assuming an initial capacity of the batteries of 2500mAh, the system takes about 25 hours to charge the batteries. The discharge time in the worst case is $\frac{2500 \times 1.5 \times 2}{65.4 \times 3.3} = 34$ hours.

4 Energy Production Model

We present in this section a realistic battery charge and discharge model which is derived from the energy harvesting system presented in Section 3. For simplicity, we assume that the solar cell is used to directly charge the batteries of the sensor. Thus, we need to formulate first the energy production function from the solar cell and second the charge and discharge model based on the sensor energy requirements.

To compute the energy production of the solar cell we take into account the relevant parameters of the solar cell (η, S) and the average monthly solar irradiance (D) in some location. Thus, the output power is computed as $P_{out} = \eta \times D \times S$ and the average energy production E_p of the solar cell is P_{out}/V_{ld} . Figure 1 shows the energy production by taking the values of average monthly irradiance in Madrid.

However, to obtain the instantaneous irradiance of the solar cell at time t , we should add a correction factor de-

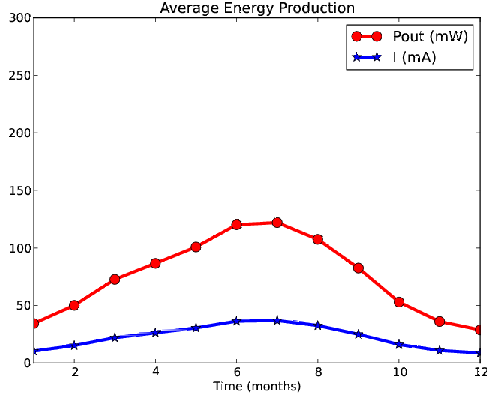


Figure 1. Average monthly output power and energy production taking the average monthly solar irradiance in Madrid.

pending on the current day and solar time. For example, in hours with no solar light the energy production is close to zero. Thus, at time t the instantaneous irradiance can be expressed as $D(t) = \frac{(t-h)^2}{4p} + k$, where t is any hour between 0:00 and 23:00, h is the hour of maximum solar light (12:00am), k is equal to the average irradiance in a month (D_{avg}) and $p = 9/k$, where 9 is the number of standard hours of solar light, which depends on the month of the year. As shown, the irradiance at time t is a quadratic function which draws a parabola with vertex at 12:00 in the noon. By taking different values of irradiance we can obtain the instantaneous irradiance in any hour of the day, as shown in Figure 2. Then, the estimated energy production at time t is $E_p(t) = \frac{D(t) \times \eta \times S}{V_{td}}$.

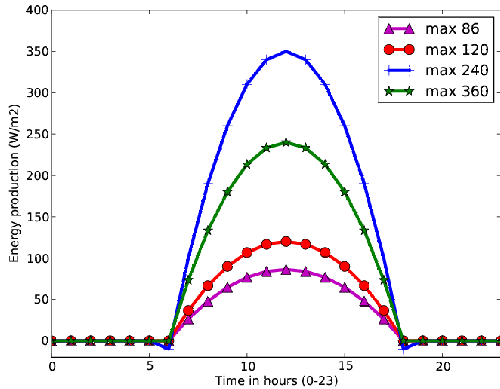


Figure 2. Irradiance per hour for different k .

4.1 Battery Discharge and Re-charge Model

The battery discharge model depends mainly on the power requirements of the node, in our case MicaZ. As explained, MicaZ is powered on by two batteries of 1.5V

with capacity ranging between 1500-3000mAh. To illustrate how these batteries are drained, we consider three different average current draws (27.7, 50, and 80 mA), which match with different operation levels in the sensor. Figure 3 on the left illustrates the discharge curves for every current draw and a battery capacity of 2500mAh.

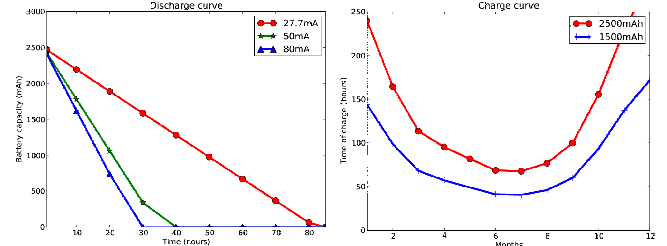


Figure 3. Battery discharge and charge curves.

In the system proposed, where the batteries are charged from the solar cell, they experiment an increase of the stored energy according to the energy production. Figure 3 on the right shows the time of charge (in hours) required to completely charge a battery with capacities of 1500mAh and 2500mAh assuming the energy production obtained from the average monthly irradiance in Madrid (see Figure 1).

5 System Model

We consider a WSN for outdoor monitoring applications, where the sensors are powered by batteries that are recharged by solar cells. Sensors drain the energy from their batteries accordingly to the application they are executing. The application defines an energy consumption pattern, which is characterized by the use in the time of its hardware components in the different operational states and by the energy consumption (or current draw) that each component consumes in each state.

In our model a sensor is represented as a triple composed of a set of components, set of states for each component, and energy consumption for each component in each state per unit of time. Let u be a sensor defined as $\langle C, \Sigma, E \rangle$, where C is the set of hardware components of u . $\forall c \in C$ let $S(c) \in \Sigma$ be the set of valid states of c , and, finally, $E(u) = \{e(c, s) \mid \forall c \in C \text{ and } s \in S(c)\}$ where $e(c, s)$ is the energy consumed by component c in state s per unit of time.

A task T executed by sensor u is a pair $T = \langle \sigma, t \rangle$, where t is the length of the period during which T is executed, $\sigma = \langle s_{c_1}, \dots, s_{c_n} \rangle$, and for each $c_i \in C$, s_{c_i} is the state of component c_i during task T . Let us define an application A as a possibly infinite sequence of tasks $A = T_1, T_2, \dots$ where for each tasks T_i and T_{i+1} are consecutive. A prefix A_τ of A is the sequence of tasks of A executed in the period

$[0, \tau]$. For a given task $T = \langle \langle s_{c_1}, \dots, s_{c_n} \rangle, t \rangle$, the energy consumption of a sensor u executing T is given by:

$$E_T = \sum_{i=1 \dots n} t \times E(c_i, s_{c_i})$$

hence, the energy consumed by sensor u during an application $A_\tau = T_1, \dots, T_m$ that lasts a time τ is given by:

$$E_{A_\tau} = \sum_{j=1 \dots m} E_{T_j}$$

Let B_{max} be the maximum battery capacity and B_{min} be the minimum battery capacity necessary to maintain the sensor working.

Consider a sensor connected to our solar energy harvesting system as explained in Section 3. The solar cell obtains the energy from the sun to charge the sensor batteries accordingly to the light conditions at each moment. Thus, the energy available in the batteries may increase (or decrease) depending on the solar irradiance received in a place and instant of time. Here, the system could be compared to a producer-consumer system, where the producer is the solar cell and the consumer is the sensor.

Let us consider a sensor u that starts executing an application A at time 0 with the battery at capacity B . At time τ the energy consumed by u is E_{A_τ} . Let also $e_p(t)$ be the energy produced by the solar cell at time t ($t \leq \tau$). Then, we can define the total energy produced by the solar cell in the interval $[0, \tau]$ as:

$$E_p(t) = \int_0^\tau e_p(t) dt$$

Analogously, letting $e_e(t)$ be the estimation of the expected energy to be produced by the solar cell at time $t > \tau$, the total energy that is estimated to be produced by the solar cell in an interval $[\tau, \tau + \Delta]$ is:

$$E_e(\tau, \tau + \Delta) = \int_\tau^{\tau+\Delta} e_e(t) dt$$

Taking the balance between the produced and consumed energy at time t and assuming a battery with infinity capacity, the battery level of B for any $t \in [0, \tau]$ can be expressed as:

$$B(t) = B(0) + E_p(t) - E_{A_t}$$

Taking the assumptions of above, let us consider any application A which is monitoring some environmental phenomena. We assume that A has several versions $A^1, A^2 \dots A^v$ each corresponding to a different level of activity and, from the point of view of the user, to a different quality level $q^i \forall i = 1 \dots v$. Note that the definition of the versions is specific to the application domain. For example, a monitoring application may have three versions: A^1

represents the highest level of functionality (its components operate at maximum duty cycle), A^2 represents an intermediate level of functionality (its components operate at a reduced duty cycle), and A^3 represents not functionality (all the components are set to Idle state).

Let us define scheduler as the subsystem that determines the execution order of the applications versions. Thus, a scheduling P is a sequence of pairs $\langle A^i, t^i \rangle$ ($1 \leq i \leq k$, for a given k) in a given interval $[0, \tau]$, where $\sum_{i=1 \dots k} t^i = \tau$. Then, the energy consumption of P in the period τ is:

$$E_\tau(P) = \sum_{i=1 \dots k} E_{A^i} \times t^i$$

The scheduler may switch among the different versions depending on the sensor energy budget and on the expected energy production, in order to maximize some QoS related metric. Specifically, we define the QoS of an application as the expression that represents the best application performance from the user perspective. For example QoS could be defined as the percentage of time in which the sensor executes A^1 or as the average of the quality levels achieved during a period of time.

6 Representing Applications in our Model

Consider a sample application A developed for the sensor u with the goal of sampling its temperature transducer every minute and sending the measured value to the sink. As explained, u is defined by the triple $\langle C, \Sigma, E \rangle$ where C includes processor & memory subsystem, timer subsystem, radio subsystem, and temperature acquisition subsystem, i.e., $C = \{Proc, Timer, Radio, Temp\}$ and $\Sigma = \{S_{Processor}, S_{Memory}, S_{Radio}, S_{Temp}\}$, where:

$$\begin{aligned} S_{Proc} &= \{Active, Sleep, Idle\} \\ S_{Timer} &= \{On, Off\} \\ S_{Radio} &= \{TX, RX, Idle, Sleep\} \\ S_{Temp} &= \{On, Off\} \end{aligned}$$

To perform its activity, A executes cyclically three tasks T_1, T_2 and T_3 which repeat indefinitely in the time, i.e. $A = \langle T_1, T_2, T_3, T_1, T_2, T_3, T_1 \dots \rangle$. Every task does the following: T_1 samples the temperature sensor, T_2 composes a message and forwards it to the network and T_3 waits for the next sampling interval. To this end, T_1 uses processor and temperature acquisition components, T_2 uses processor and radio components, and T_3 uses just the timer subsystem. In our model, every task is represented by $\langle \sigma, t \rangle$ where σ is the set of energy states that uses each component and t its execution time. In this example:

It is immediate that this application has a duty cycle equal to 50%. Consider now three versions of A , A^1, A^2 ,

$$\begin{aligned}
T_1 &= \langle \langle S_{Proc}=Active, S_{Timer}=Off, S_{Radio}=Sleep, S_{Temp}=On \rangle, 10 \rangle \\
T_2 &= \langle \langle S_{Proc}=Active, S_{Timer}=Off, S_{Radio}=TX, S_{Temp}=Off \rangle, 15 \rangle \\
T_3 &= \langle \langle S_{Proc}=Idle, S_{Timer}=On, S_{Radio}=Sleep, S_{Temp}=Off \rangle, 25 \rangle
\end{aligned}$$

and A^3 each one using a different duty cycle, say 50%, 25% and 1% respectively. Therefore, every version uses the tasks, components, and energy states specified for A but the interval in which they are executed is reduced accordingly to its duty cycle. Specifically, task T_3 of A^1 , A^2 and A^3 last 25%, 12.5% and 0.5% of the total time, respectively. The user of the application assigns to each version a quality level q that is defined according to the user preferences. It is important to stress that the quality levels are statically defined and they cannot vary. For example, in a scale from 1 to 10, it might be $q^1 = 10$, $q^2 = 5$ and $q^3 = 0$. Assume that QoS metric selected by the user is the average quality level during the period. If we execute A^1 during 1/2 of the period and A^2 during 1/4 of the period, then the QoS of A becomes $QoS(A) = q^1 \times \frac{1}{2} + q^2 \times \frac{1}{4} + q^3 \times \frac{1}{4}$.

7 Adaptive QoS by Graceful Degradation

We present a novel scheduling strategy for solar cells powered sensors aimed at the maximization of the QoS of a WSN application. This strategy selects the versions of the application that optimize the QoS according to the available energy budget and to the expected energy production. Consider an application A with v versions $A^1, A^2, \dots, A^v$ each one with a quality level $q^1, q^2, \dots, q^v$. A QoS metric of A is a linear function that expresses the best performance of A during a period of time τ , expressed as:

$$QoS(A) = q^1 X^1 + q^2 X^2 + \dots + q^v X^v$$

where $X^1, X^2, \dots, X^v$ are the variables of the system that represent the overall execution times of $A^1, A^2, \dots, A^v$ respectively. In order to limit the solutions space, we add some constraints to the system:

- $X^i \geq 0 \forall i \in [1 \dots v]$.
- $\sum_{i=1 \dots v} X^i = \tau$.
- Given any scheduling P of A , and an initial battery level $B(0)$, must be: $B_{max} \geq B(0) + E_p(t) - E_t(P) \geq B_{min} \forall t > 0$, i.e. the battery level should always be included between B_{max} and B_{min} .

Therefore, the goal is to find the optimal values of $X^1, X^2, \dots, X^v$ that maximize $QoS(A)$ subject to the set of defined constraints. The system above described is a typical problem of linear programming where the solution, if exists, is achieved by mean of some iterative searching in the space of solutions up to find the optimal. Solving this

problem in a sensor is very challenging due to processing and memory requirements for this solution. Furthermore, optimizing $X^1, X^2, \dots, X^v$ is not sufficient as the sensors would also have to find an appropriate scheduling giving $X^1, \dots, X^v$.

As an alternative, we propose a scheduler that implements a graceful degradation policy to maximize a QoS metric. The scheduler is executed at the end of each duty cycle, and it selects a version of the application to be executed in the next duty cycle, according to the QoS metric, the estimation of the battery level, and the expected energy production in the next duty cycle. Specifically, let us assume that a time t the sensor u has a battery level equal to $B(t)$. Consider the version A^i of the application, with duty cycle Δ_i and assume that the expected production in the next period is $E_e(t, t + \Delta_i)$. Then, if the scheduler selects for the execution the version A^i of the application, the expected battery level at time $t + \Delta_i$ is given by: $B_i(t + \Delta_i) = B(t) - E_{A^i, \Delta_i} + E_e(t, t + \Delta_i)$.

Let $th_1, th_2, \dots, th_{v-1}$ be a set of thresholds on the battery level such that $th_i > th_{i+1} \forall 1 \leq v - 1$. If there exists an $i \in [1, v - 1]$ such that $B_i(t + \Delta_i) \geq th_i$ and $B_j(t + \Delta_j) < th_j \forall j < i$ then the scheduler selects A^i as the version to be executed in the next interval. Otherwise the scheduler executes A^v .

8 Evaluation of Thresholds

The selection of the thresholds is a critical aspect of our strategy that impacts on the final QoS of the applications. We evaluated by simulation different thresholds with the goal of maximizing the QoS metric expressed by the user according to the estimated energy production from the cell.

We have evaluated our strategy by using a simulator developed in C. We considered a WSN application A to be deployed on a MicaZ platform (see Table 2) with 4 versions A^1, A^2, A^3, A^4 , each one with a different duty cycle (DC): 50%, 30%, 10%, 3% respectively. For each version, we assumed a different energy consumption in active mode: 40, 25, 15 and 7.5 milliamperes. As described, the energy consumption for each version depends on the length of the active period. For example, according to the Low Power Listening (LPL) implementation described in [1], a $DC = 1\%$ has a sleep period of 495ms and an active period of 5ms. Note that a $DC = 100\%$ represents no sleep periods. This means that, when A^4 is executing, during the active period it consumes 7.5 milliamperes per unit of time, A^3 consumes 15 milliamperes, and so on. We also took into account the energy consumption during the sleep period, which corresponds to 35μA per second (the radio and the microcontroller are in idle mode). The lifetime of the four versions when there exist no energy harvesting system is depicted in Figure 4. We show in the x axis the time of

simulation starting at 1st of January, 2012. With a very low DC=3%, theoretically A^4 may execute until July, 2013.

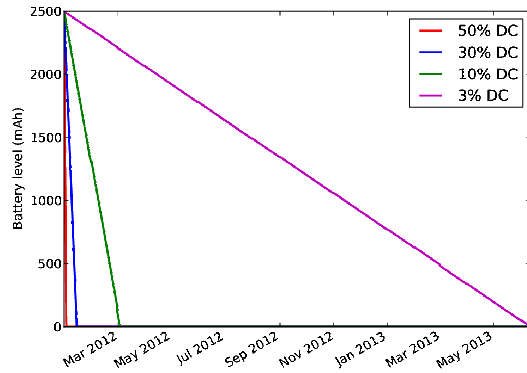


Figure 4. Lifetime of the four versions using different DC when no solar cell is used.

These results can be meaningfully improved by connecting a solar cell to the MicaZ platform, as described in Section 3. We considered the average monthly solar irradiance in Madrid (data provided by RetScreen [13]): 84.58, 123.33, 178.75, 212.91, 247.91, 300, 264.16, 202.91, 130.41, 88.75, 70.83 W/m^2 . To estimate the instantaneous energy production we took into account the light hours as explained in Section 4. Additionally, to do more realistic simulations we varied the standard light hours as well as the first and last light hour accordingly to each month. The simulations started the 1st of January, 2012 at 0:00am with the version A^1 . We evaluated five different configurations of battery thresholds varying between 500 and 2000mAh. Figure 5 shows the results of lifetime and battery level for three of these five configurations.

Regardless to the configuration, the solar cell may prolong the lifetime of the application until infinite or until the physical duration of the battery (we present just periods of 5 years). As shown, our scheduler allows to dynamically interchange the version of the application when the battery level achieves a determined threshold: the scheduler selects A^1 when the battery level is above than threshold 1, A^2 when the battery level is above than threshold 2 and below than threshold 1, A^3 when the battery level is above than threshold 3 and below than threshold 2 and, finally, it selects A^4 when the battery level is below than threshold 3. As the energy production is constant, the battery level draws a pattern which is repeated each year: initially, when the most energy consuming version is executing (A^1) and corresponding to a month of the year with a very low energy production (January), the residual battery level drops very quickly below the last threshold. This happens because of the energy consumption of (A^1) is bigger than the production in this month. The abrupt reduction of the battery level at those points of the year is due to the fact that in winter the

Thresholds	A^1	A^2	A^3	A^4	QoS (%)
2250,1750,1250	54.89	1103.8	626.16	15.15	86.65
2000,1500,1000	55.12	1103.8	626.19	14.89	86.66
1875,1250,625	54.53	1108.94	625.96	10.57	86.7
1500,750,625	55.6	1103.8	626.25	14.36	86.67
1250,750,625	55.83	1106.4	613.49	24.28	86.63

Table 3. Execution times (in days) and QoS results (%) for each threshold configuration.

production is significantly lower and insufficient to meet the QoS requirements. When this happens, the lowest energy consuming version (A^4) gets executing until the energy production is bigger than the energy consumption and then, the battery level starts to grow corresponding to the months of spring and summer, until executing (A^2). As shown, during the summer season the scheduler alternatively selects A^1 and A^2 depending on the solar light hours. During the autumn and winter season, the battery level drops again, and the scheduler alternatively selects A^2 and A^3 . As shown, the battery level is always maintained above the last threshold. In general, the experiments show that the battery level can be recovered to be above the second threshold and, subsequently, higher is this threshold higher is the battery level. Finally, Table 3 shows for each configuration of thresholds the execution times (in days) for A^1 , A^2 , A^3 , A^4 in the columns from 2 to 5 respectively, and the results of QoS assuming the quality levels 100, 90, 80, 70 (in percentage) in column 6. According to these quality levels, the third threshold configuration (1875,1250,625) gets the best QoS. A comparison of the lifetime of the four versions against such a configuration is shown in Figure 6.

9 Conclusions and Future Work

Energy harvesting systems (e.g. solar cells) allow to provide potentially infinite sensor's lifetime. However, in practice, the production of these systems is intermittent and insufficient to constantly satisfy the power demand of WSN applications. In this paper we present a solar cell-based system to fulfill the energy requirements of the sensor nodes. We used this system to propose a scheduler that maximizes the QoS of WSN applications. Our scheduler selects dynamically the most suitable version of the application to be executed according the estimated energy production from the solar cell, an expression of the QoS defined by the user, and the energy consumption. We have studied the impact of selecting different threshold configurations on the QoS of WSN applications. The results show that our approach may prolong the applications lifetime until infinite, only limited by the physical duration of the batteries. As future work we plan to dynamically adapt the sampling frequency of the

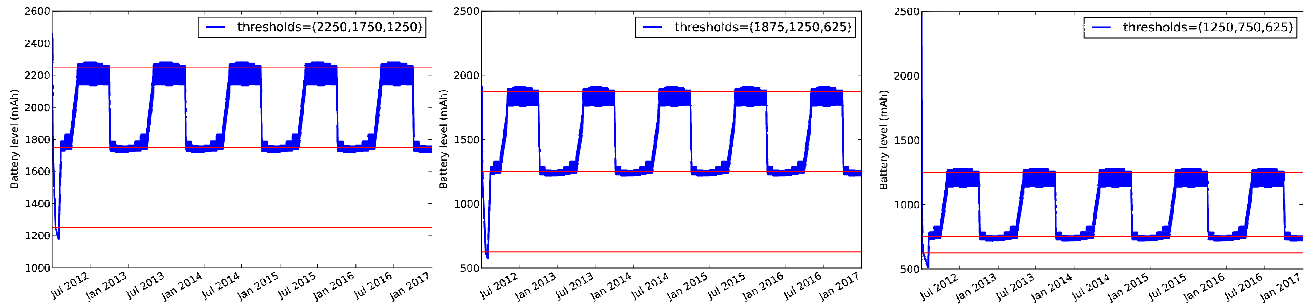


Figure 5. Lifetime and battery level for three different configurations of thresholds.

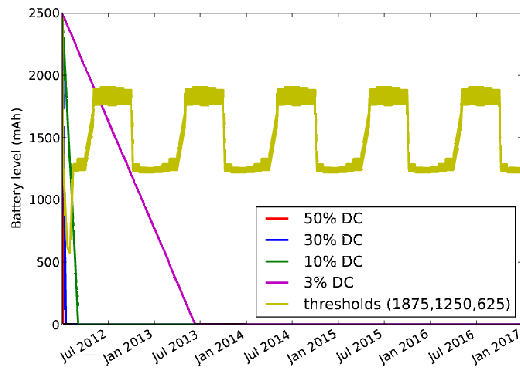


Figure 6. Lifetime comparison of the four versions and for the thresholds=(1875,1250,625).

WSN applications according to the estimated energy production and computing the solar cells requirements based on the power demand of sensors. Another work consists in to define empirically the thresholds used.

References

- [1] T. 2.x Documentation. Cc2420 radio stack, 2007.
- [2] K. Akkaya and M. Younis. Energy and qos aware routing in wireless sensor networks. *Cluster Computing*, 8:179–188, July 2005.
- [3] C. Alippi, G. Anastasi, M. Di Francesco, and M. Roveri. Energy management in wireless sensor networks with energy-hungry sensors. *Instrumentation Measurement Magazine, IEEE*, 12(2):16–23, april 2009.
- [4] L. Benini, A. Bogliolo, and G. De Micheli. A survey of design techniques for system-level dynamic power management. *Very Large Scale Integration (VLSI) Systems, IEEE Transactions on*, 8(3):299–316, june 2000.
- [5] D. Chen and P. K. Varshney. QoS Support in Wireless Sensor Networks: A Survey. In *Proc. Int. Conference on Wireless Networks, Las Vegas, Nevada, USA*, June 2004.
- [6] W. chun Feng, X. Feng, and R. Ce. Green supercomputing comes of age. *IT Professional*, 10(1):17–23, jan.-feb. 2008.
- [7] F. Components. Bp solar - msx-005f - solar panel, 2011.
- [8] M. Cooperation. Micaz datasheet, 2010.
- [9] E. Felemban, C.-G. Lee, and E. Ekici. Mmspeed: multipath multi-speed protocol for qos guarantee of reliability and. timeliness in wireless sensor networks. *Mobile Computing, IEEE Transactions on*, 5(6):738–754, june 2006.
- [10] T. He, J. Stankovic, C. Lu, and T. Abdelzaher. Speed: a stateless protocol for real-time communication in sensor networks. In *Distributed Computing Systems. Proc. 23rd International Conference on*, pages 46–55, may 2003.
- [11] R. Iyer and L. Kleinrock. Qos control for sensor networks. In *Communications, 2003. ICC '03. IEEE International Conference on*, volume 1, pages 517–521 vol.1, may 2003.
- [12] C. Lin. Qos routing in ad hoc wireless networks. In *Local Computer Networks, 1998. LCN '98. Proc. 23rd Annual Conference on*, pages 31–40, oct 1998.
- [13] N. S. meteorology and S. Energy. RetScreen. <http://eosweb.larc.nasa.gov/sse/RETScreen/>, 2011.
- [14] S. Murugesan. Harnessing green it: Principles and practices. *IT Professional*, 10(1):24–33, jan.-feb. 2008.
- [15] V. Raghunathan, A. Kansal, J. Hsu, J. Friedman, and M. Srivastava. Design considerations for solar energy harvesting wireless embedded systems. In *Proc. 4th Int. Sym. on Information processing in sensor networks, IPSN '05*, Piscataway, NJ, USA, 2005. IEEE Press.
- [16] F. Salvadori, M. de Campos, P. Sausen, R. de Camargo, C. Gehrke, C. Rech, M. Spohn, and A. Oliveira. Monitoring in industrial systems using wireless sensor network with dynamic power management. *Instrumentation and Measurement, IEEE Transactions on*, 58(9):3104–3111, sept. 2009.
- [17] P. S. Sausen, J. R. de Brito Sousa, M. A. Spohn, A. Perkusich, and A. M. N. Lima. Dynamic power management with scheduled switching modes. *Comput. Commun.*, 31:3625–3637, September 2008.
- [18] A. Sinha and A. Chandrakasan. Dynamic power management in wireless sensor networks. *Design Test of Computers, IEEE*, 18(2):62–74, mar/apr 2001.
- [19] K. Sohrabi, J. Gao, V. Ailawadhi, and G. Pottie. Protocols for self-organization of a wireless sensor network. *Personal Communications, IEEE*, 7(5):16–27, oct 2000.
- [20] F. Xia. Qos challenges and opportunities in wireless sensor/actuator networks. *Sensors*, 8(2):1099–1110, 2008.
- [21] M. A. Yigitel, O. D. Incel, and C. Ersoy. Qos-aware mac protocols for wireless sensor networks: A survey. *Comput. Netw.*, 55:1982–2004, June 2011.
- [22] C. Zhu and M. Corson. Qos routing for mobile ad hoc networks. In *INFOCOM 2002. Twenty-First Annual Joint Conference of the IEEE Computer and Communications Societies*, volume 2, pages 958–967 vol.2, 2002.